

第 7 章

带上X光眼镜测试软件

第二部分“测试基础”进行到这里，我们已经学会了4个基本测试技术之中的3个：静态黑盒（测试产品说明书）、动态黑盒（测试软件）和静态白盒（检查程序代码）。本章我们将学习第4个基本测试技术——动态白盒测试。在测试软件时，我们将利用X光眼镜洞察软件的“盒子”里面。

除了X光本身的含义，还要充当程序员——如果能够胜任的话。如果没有，也不要害怕。本章使用的示例并不复杂，花一点时间就可以领会。即使只掌握此类测试的皮毛，都会成为更高效的黑盒测试员。

如果具有一些编程经验，本章可以作为步入广阔测试领域的敲门砖。大多数公司聘用软件测试员主要是为了进行软件的底层测试。他们在寻找同时具有编程和测试技术的人，这是难得一见的。

本章的重点包括：

- 什么是动态白盒测试
- 调试和动态白盒测试之间的区别
- 单元和集成测试是什么
- 如何测试底层功能
- 底层测试所需的数据范围
- 如何强制软件以某种方式运行
- 衡量测试完整性的各种方法

7.1 动态白盒测试

至此，我们应该相当熟悉静态、动态、白盒和黑盒等术语了。既然本章是关于动态白盒测试，就一定会讲述关于该技术的内容。由于是动态的，就一定是测试运行中的程序，由于是白盒，就一定要洞察盒子里面，检查代码并观察运行状况。这好像是带上X光眼镜测试软件。

用一句话来概括，动态白盒测试是指利用查看代码功能（做什么）和实现方式（怎么做）得到的信息来确定哪些需要测试、哪些不要测试、如何开展测试。动态白盒测试的另一个常用名称是结构化测试（structural testing），因为软件测试员可以查看并使用代码的内部结构，从而设计和执行测试。

为什么了解盒子内部情况和软件工作方式有好处呢？参见图7-1。该图给出了执行加减乘除基本运算操作的两个盒子。

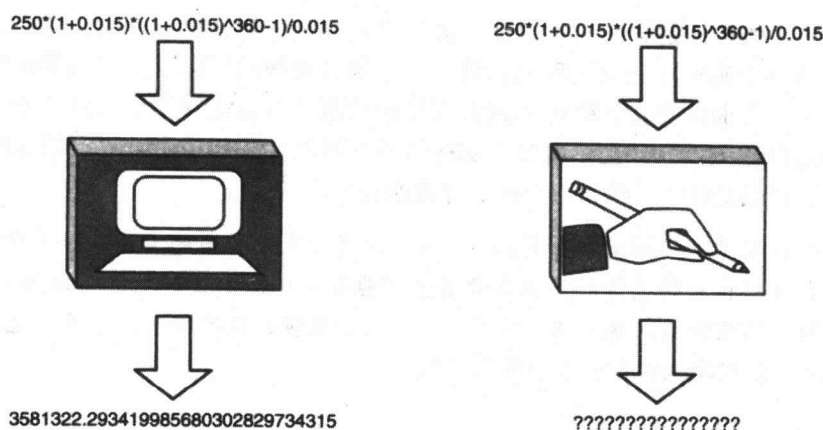


图7-1 假如知道一个盒子包含一台计算机，而另一个盒子是人用纸笔计算，就会选择不同的测试用例

如果不知道盒子的工作方式，就会运用第5章“带上眼罩测试软件”所讲的动态黑盒测试技术。但是，如果看到盒子内部，发现一个盒子包含一台计算机，而另一个盒子包含拿纸笔的人，就会选择完全不同的测试方法。当然，这是一个过于简单化的例子，但是它恰如其分地说明了解软件的运作方式对测试手段的影响。

动态白盒测试不仅仅是查看代码的运行情况，还包括直接测试和控制软件。动态白盒测试包括以下4个部分：

- 直接测试底层函数、过程、子程序和库。在Microsoft Windows 中这称为应用程序编程接口（API）。
- 以完整程序的方式从顶层测试软件，但是根据对软件运行的了解调整测试用例。
- 从软件获得读取变量和状态信息的访问权，以便确定测试与预期结果是否相符，同时，强制软件以正常测试难以实现的方式运行。
- 估算执行测试时“命中”的代码量和具体代码，然后调整测试，去掉多余的测试用例，补充遗漏的用例。

本章余下部分讨论以上各部分。阅读的同时，请思考怎样用它们来测试我们熟悉的软件。

7.2 动态白盒测试和调试

一定不要把动态白盒测试和调试（debugging）弄混了。做过编程的人可能会花费大量时间调试自己编写的代码。这两项技术表面上很相似，因为它们都包括处理软件缺陷和查看代码的过程，但是它们的目标大不相同（见图7-2）。

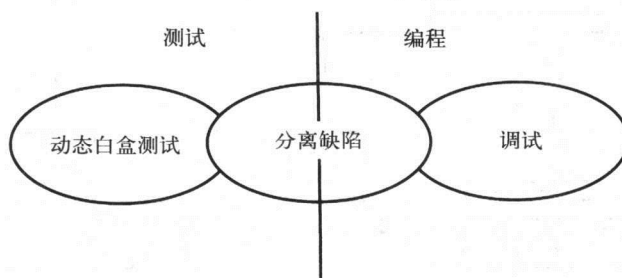


图7-2 动态白盒测试和调试有不同的目标，但是其中有交叉现象

动态白盒测试的目标是寻找软件缺陷，调试的目标是修复缺陷。然而，它们在隔离软件缺陷的位置和原因上确实存在交叉现象。第19章“报告发现的问题”将详细讲述这一点，但眼前要考虑的是这方面的交叉。软件测试员应该把问题缩减为能够演示软件缺陷的最简化测试用例。如果是白盒测试，甚至还要包括那些值得怀疑的代码行信息。进行调试的程序员从这里继续，判断到底是什么导致软件缺陷，并设法修复。

注意 执行这些底层的测试，会用到许多和程序员使用的相同的工具。如果程序编译过，可能会使用相同的编译器，但是设置上可能不一样，以更好地检测缺陷为目标。可能会使用代码级的调试器来单步跟踪程序，观察变量，设置断点，等等。也可能自己编写程序来分别测试需要检查的模块代码。

7.3 分段测试

回顾第2章“软件开发的过程”所讲的几种软件开发模式。大爆炸模式是最简单的，但也是最混乱的。所有的程序一下子堆在一块，期待奇迹出现。开发小组希冀各部分都没问题，并且会形成一个产品。现在可以断言，在这种模式下进行测试是非常困难的。最多可以进行动态黑盒测试，把这个准产品当做一团烂泥并在其中摸索着看能找出点什么。

我们已经知道这种测试的费用很高，因为软件缺陷是在最后关头才发现的。从测试的角度看，产生高额费用有如下两个原因：

- 难以，有时甚至不可能找出导致问题的原因。软件好比是一台无法工作的笨重机器——在一侧投下一个球，但是另一侧并没有出现黄油吐司和热咖啡，无法知道是哪个小零件坏了，导致整个装置的失败。
- 某些软件缺陷掩盖了其他软件缺陷。测试可能失败。程序员自行调试问题并修复，但是当重新测试时，软件依然失败。太多问题一个摞一个，核心错误很难弄清。

7.3.1 单元测试和集成测试

解决上述麻烦的方法当然是开始就不让它发生。如果代码分段构建和测试，最后合在一起形成更大的部分，那么整个产品无疑会链接在一起（见图7-3）。

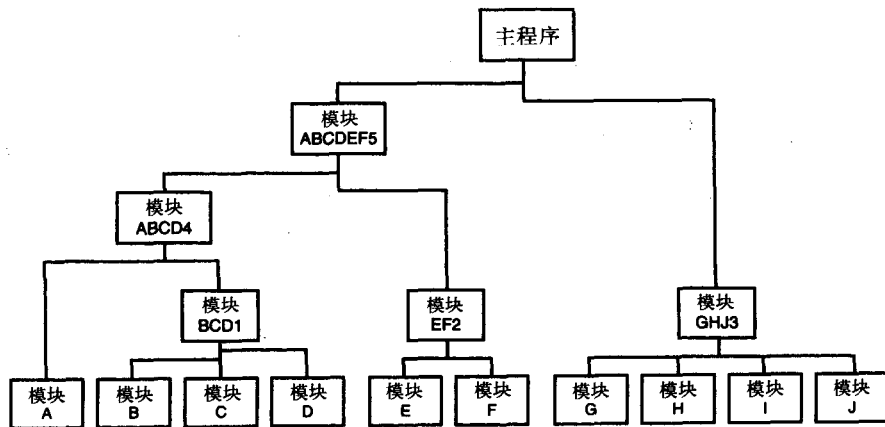


图7-3 独立代码段分别建立和测试，然后集成并再测试

在底层进行的测试称为单元测试 (unit testing) 或者模块测试 (module testing)。单元经过测试, 底层软件缺陷被找出并修复之后, 就集成在一起, 对模块的组合进行集成测试 (integration testing)。这个不断增加的测试过程继续进行, 加入越来越多的软件片段, 直至整个产品——至少是产品的主要部分——在称为系统测试 (system testing) 的过程中一起测试。

采取这种测试策略很容易隔离软件缺陷。在单元级发现问题时, 问题肯定就在那个单元中。如果在多个单元集成时发现软件缺陷, 那么它一定与模块之间的交互有关。当然也有例外, 但是总的说来, 测试和调试比一起测试所有内容要有效得多。

这种递增测试有两条途径: 自底向上 (bottom-up) 和自顶向下 (top-down)。在自底向上测试中 (见图7-4), 要编写称为测试驱动模块调用正在测试的模块。测试驱动模块以和将来真正模块同样的方式挂接, 向处于测试的模块发送测试用例数据, 接受返回结果, 验证结果是否正确。采取这种方式, 可以对整个软件进行非常全面的测试, 为它提供全部类型和数量的数据, 甚至高层难以发送的数据。

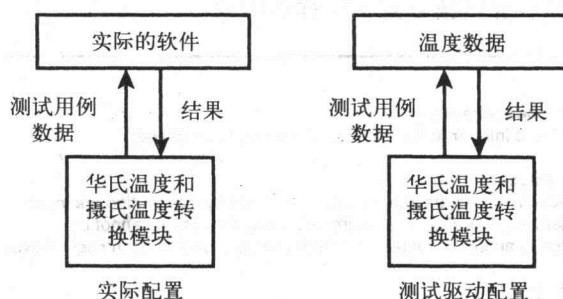


图7-4 测试驱动可以代替实际软件, 对底层模块进行更有效的测试

自顶向下测试有点像小规模的大爆炸测试。毕竟, 如果更高层软件已经完成, 测试其下层的模块就一定太晚了, 对吗? 其实并不完全正确。请看图7-5。在该例中, 底层接口模块用于从电子温度计采集温度数据。显示模块正好在此接口模块的上面, 从该接口读取数据, 向用户显示结果。为了测试高层显示模块, 就需要借助火把、水、冰和深度的冻结来改变传感器的温度, 并将数据传递到导线上。

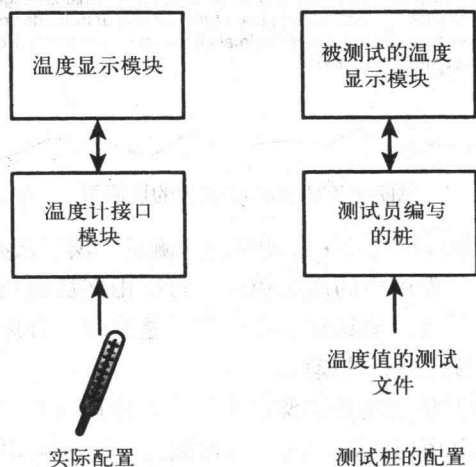


图7-5 测试桩向处于测试的模块发送数据

与其通过设法控制温度计的温度来测试温度显示模块，不如编写一小段称为桩（Stub）的代码充当接口模块，从文件中把温度值直接提供给显示模块。显示模块将会读取数据并显示温度，就像直接从实际温度计接口模块中读取数据一样。它不可能有何不同。有了这个测试桩配置，就可以快速从头到尾试验各种测试值，验证显示模块的操作。

7.3.2 单元测试示例

许多编译器会提供一个把ASCII字符转换为整数值常用函数。

该函数接收一串带正负号的数字或者空格、字母等特殊字符，将其转换为数值——例如字符“12345”转换为数字12, 345。这是用于处理用户在对话框中输入值的一个相当常用的函数——例如，某人的年龄或者财务账目。

执行该操作的C语言函数是atoi()，表示“把ASCII字符转换为整数”。图7-6给出了该函数的说明。如果不是C程序员，也不需气馁。除了第1行是说明如何调用函数的，其他部分说明采用英语，可以用来为任何计算机语言定义同样的函数。

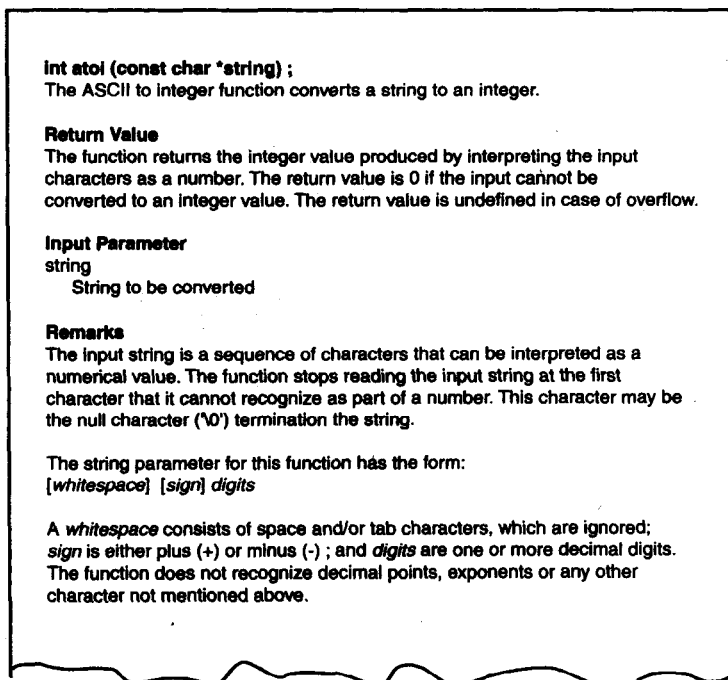


图7-6 C语言atoi()函数的说明页

如果作为软件测试员受命对该模块进行动态白盒测试，该怎么办？

首先可以确定该模块属于程序中的底层模块，可以由高层模块调用，但是自己不能调用其他模块。通过查看内部代码可以确认这一点。如果是这样，合理的做法是编写一个测试驱动以独立于程序其他部分的形式测验该模块。

测试驱动将向atoi()函数发送创建好的测试字符串，读取这些字符串的返回值，与预期结果相比较。测试驱动完全可以用与函数相同的语言编写——在本例中，用C语言——但是用其他语言编写也可以，只要能够与测试模块接口即可。

测试驱动可以有多种形式，可以是如图7-7所示的简单对话框，用于输入测试字符串并查看结果；也可以是从文件读取测试字符串和预期结果的独立程序。由用户驱动的对话框的交互性和灵活性强——可以用来进行黑盒测试。而独立驱动可以极其快速地直接从文件读写测试用例。

下一步要分析说明书，确定应该采用的黑盒测试用例，然后运用等价类划分技术减少测试用例集合（还记得第5章吗？）。表7-1列出了一些包含输入字符串和预期输出值的测试用例。该表不是完整的清单。

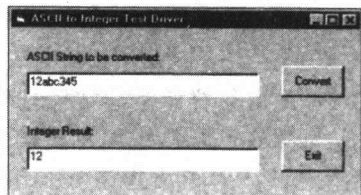


图7-7 对话框测试驱动可以用于向被测试的模块发送测试用例

表7-1 ASCII字符转换为整数的测试用例示例

输入字符串	输出整数值	输入字符串	输出整数值
"1"	1	"1.2"	1
"-1"	-1	"2-3"	2
"+1"	1	"abc"	0
"0"	0	"a123"	0
"-0"	0	等	
"+0"	0		

最后，研究代码看函数是如何实现的，利用模块的白盒知识增减测试用例。

注意 在进行白盒测试之前，一定要根据说明书建立黑盒测试用例。用这种方式可以真正测试模块的功能和作用。如果先从模块的白盒角度建立测试用例，检查代码，就会偏向于以模块工作方式建立测试用例。程序员或许误解了说明，于是测试用例就会不对。虽然测试用例精确完整地测试了模块，但是可能不准确，因为没有测试预期的操作。

根据白盒知识增减测试用例其实是根据程序内部的信息对等价划分的进一步提炼。原来的黑盒测试用例可能认为内部ASCII表会把“a123”和“z123”当做不同且重要的等价类。经过检查软件之后，就会发现程序员不管ASCII表，而只检查数字、正负号和空格。基于此，应该决定删去其中一些用例，因为这两者都属于同一个等价类。

通过仔细检验代码，就会发现正负号的处理可能有点问题，甚至不明白它是怎么回事。在这种情况下，可以增加一些带有正负号的测试用例来确认。

7.4 数据覆盖

前面关于atoi()函数白盒测试的例子是相当简化的，忽略了一些细节，即查看代码决定如何调整测试用例。事实上，除了仔细阅读软件找好思路之外，还有很多事要做。

合理的方法是像黑盒测试那样把软件代码分成数据和状态（或者程序流程）。从同样的角度看软件，可以相当容易地把得到的白盒信息映射到已经写完的黑盒测试用例上。

首先考虑数据。数据包括所有的变量、常量、数组、数据结构、键盘和鼠标输入、文件、屏幕输入/输出，以及调制解调器、网络等其他设备的输入和输出。

7.4.1 数据流

数据流 (Data flow) 覆盖主要是指在软件中完全跟踪一批数据。在单元测试级, 数据仅仅通过了一个模块或者函数。同样的跟踪方式可以用于多个集成模块, 甚至整个软件产品——尽管这样做是非常耗时的。

如果在底层测试函数, 就会使用调试器观察变量在程序运行时的数据 (见图7-8)。通过黑盒测试, 只能知道变量开始和结束的值。通过动态白盒测试, 还可以在程序运行期间检查变量的中间值。根据观察结果就可以决定更改某些测试用例, 保证变量取得感兴趣的、甚至具有风险的中间值。

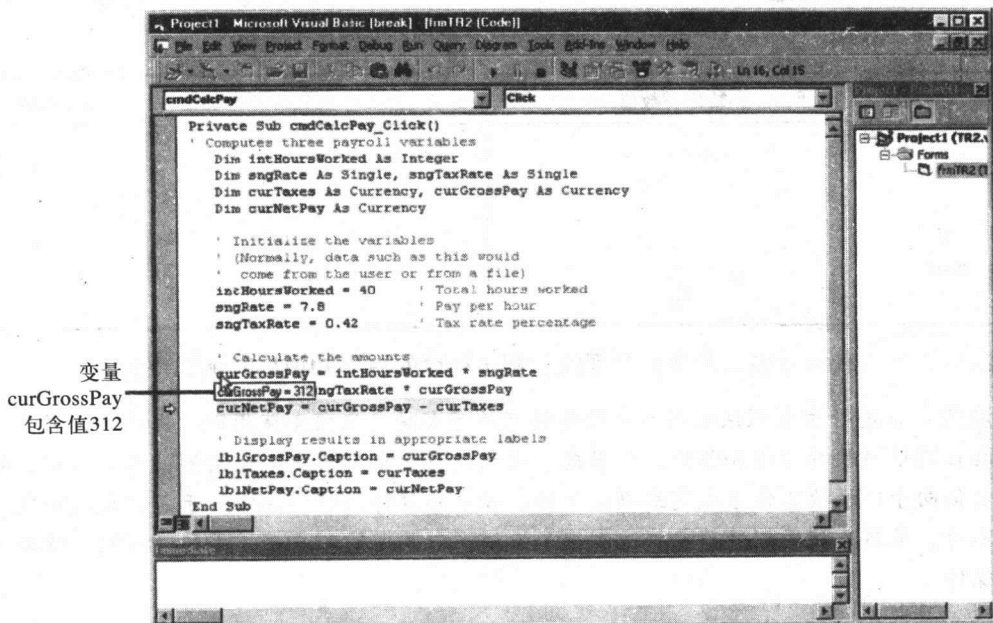


图7-8 调试器和观察变量有助于在整个程序中跟踪一个变量的值

7.4.2 次边界

次边界在第5章与ASCII表和2的幂问题一起讨论过。这些是次边界导致软件缺陷的最常见例子, 而软件的各个部分都有自己独特的次边界。以下是其他一些例子:

- 计算税收的模块在某些财务结算处可能从使用数据表转向使用公式。
- 在RAM底端运行的操作系统也许开始把数据移到硬盘上的临时存储区。这种次边界甚至无法确定, 它随着磁盘上剩余空间的数量而发生变化。
- 为了获得更高的精度, 复杂的数值分析程序根据数字大小可能切换到不同的等式以解决问题。

如果进行白盒测试, 就需要仔细检查代码, 找到次边界条件, 并建立能测试它们的测试用例。询问编写代码的程序员是否知道这些条件, 并对内部数据表给予特别的注意, 因为这里聚集了大量次边界条件。

7.4.3 公式和等式

公式和等式通常深藏于代码中，从外部看，其形式和影响不是非常明显。计算复利的财务程序一定包含以下公式：

$$A=P(1+r/n)^{nt}$$

其中

P=本金

r=年利率

n=每年复加的利率次数

t=年数

A=t年后的本息总和

优秀的黑盒测试员很可能选择n=0的测试用例，但是白盒测试员在看到代码中的公式之后，就知道这样做将导致除零错，而使公式乱套。

然而，如果n是另一项计算的结果，会怎样呢？也许软件根据用户输入来设置n的值，或者为了找出最低赔付金额而从算法角度试验各种n值。软件测试员需要考虑有没有n值为零的情形出现，指出什么样的程序输入会导致它出现。

技巧 撇开代码中的公式和等式，查看它们使用的变量，在程序正常输入和输出之外，为其建立测试用例和等价划分。

7.4.4 错误强制

本章中数据测试的最后一种类型是错误强制（error forcing）。如果执行在调试器中测试的程序，不仅能够观察到变量的值——还可以强制改变变量的值。

在进行复利计算时，如果找不到将复加数设置为零的直接方法，就可以利用调试器来强制赋值。于是软件不得不处理这情况，或者报告处理不了。

注意 在使用错误强制时，小心不要设置现实世界中不可能出现的情况。如果程序员在函数开头检查n值必须大于零，而且n值仅用于该公式中，那么将n值设为零。使程序失败的测试用例就是非法的。

如果仔细选择了错误强制情况，并和程序员一起反复检查以确认它们是合法的，错误强制就是一个有效的工具。借此可以执行其他方式难以实现的测试用例。

强制显示错误提示信息

使用错误强制的上好方法是迫使软件中的所有错误提示信息显示出来。大多数软件使用内部错误代码表示错误提示信息。当内部错误条件标志被置位时，错误处理程序读取存放错误代码的变量，在表中查找代码，并显示相应的信息。

许多错误情况是难以建立的——例如挂接2 049台打印机。但是如果只是想测试错误提示信息是否正确（拼写、语言、格式等），那么使用错误强制是最有效的查看方式。然而，要记住，这不是测试检测错误的代码，而是显示错误的代码。

7.5 代码覆盖

与黑盒测试一样，测试数据只是一半工作。为了全面地覆盖，还必须测试程序的状态以及程序流程，必须设法进入和退出每一个模块，执行每一行代码，进入软件每一条逻辑和决策分支。这种类型的测试叫做代码覆盖（code coverage）测试。

代码覆盖测试是一种动态白盒测试，因为它要求通过完全访问代码以查看运行测试用例时经过了哪些部分。

代码覆盖测试最简单的形式是利用编译环境的调试器通过单步执行程序查看代码。图7-9给出了一个运行中的Visual Basic调试器的例子。

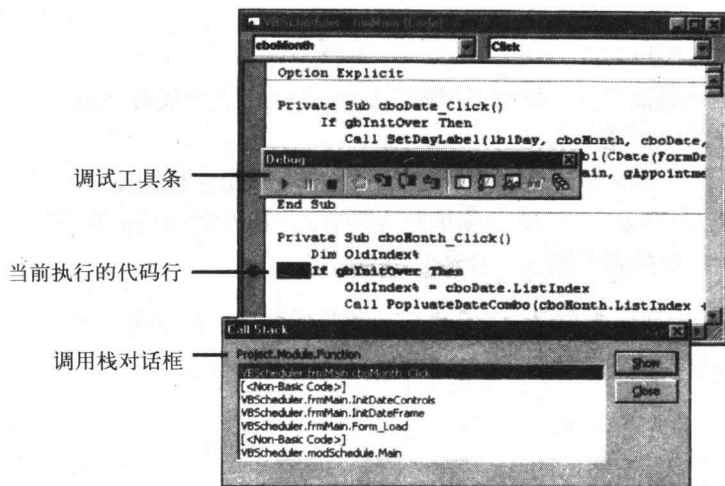


图7-9 调试器允许单步执行程序，查看运行测试用例时执行的代码行和模块

对于小程序或者单独模块，使用调试器一般就足够了。然而，对大多数程序进行代码覆盖测试要用到称为代码覆盖率分析器（code coverage analyzer）的专用工具。图7-10给出了该类工具的一个示例。

代码覆盖率测试器挂接在正在测试的软件中，当执行测试用例时在后台执行。每当执行一个函数、一行代码或一个逻辑决策分支时，分析器就记录相应的信息。从中可以获得指示软件哪些部分被执行，哪些部分未被执行的统计结果。利用该数据可以得到：

- 测试用例没有覆盖软件的哪些部分。如果某个模块中的代码从未执行，就需要额外编写测试该模块函数的用例。
- 哪些测试用例是多余的。如果执行一系列测试用例，而未增加代码覆盖率的百分比，那么这些测试用例就可能处于同一个等价划分。
- 为了使覆盖率更好，需要建立什么样的新测试用例。通过观察覆盖率低的代码，看它如何工作，做了什么，从而建立可以更彻底地测试它的新测试用例。

此外，还可以得到软件质量的大致情况。如果测试用例覆盖了软件的90%而未发现任何软件缺陷，就说明软件质量非常好。相反，如果测试只覆盖了软件的50%仍然发现了一些软件缺陷，就说明软件还要大加改进。

提示 不要忘记第3章“软件测试的实质”中讨论到的杀虫剂现象——软件测试得越多，

它对测试的免疫能力越强。如果测试用例覆盖了软件的90%而未发现任何软件缺陷,也有可能是软件的构造不好——可能是软件对测试具有了免疫能力。增加新的测试用例可能会暴露余下的10%具有非常多的缺陷。

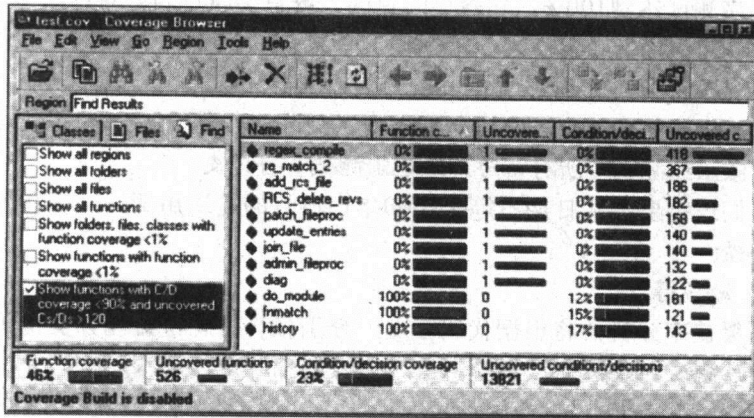


图7-10 代码覆盖分析器提供了评价测试用例效果的详细信息
(本图由Bullseye Testing Technology授权并提供)

7.5.1 程序语句和代码行覆盖

代码覆盖最直接的形式称为语句覆盖 (statement coverage) 或者代码行覆盖 (line coverage)。如果在测试软件的同时监视语句覆盖, 目标就是保证程序中每一条语句最少执行一次。对于清单7-1所示的小程序, 100%语句覆盖就是从第1行执行到第4行。

清单7-1 测试这个简单程序的每一行非常容易

```
1: PRINT "Hello World"
2: PRINT "The date is: "; Date$
3: PRINT "The time is: "; Time$
4: END
```

有人可能认为这是完全测试程序的最好方法, 可以边进行测试, 边补充测试用例, 直至程序中的每一条语句都不漏掉为止。遗憾的是语句覆盖是一种误导。可以说即使全部语句都被执行了, 但是不能说走遍了软件的所有路径。

7.5.2 分支覆盖

试图覆盖软件中的所有路径称为路径覆盖。路径测试最简单的形式称为分支覆盖测试。请看如清单7-2所示的程序。

清单7-2 IF语句在代码中建立了另外一个分支

```
1: PRINT "Hello World"
2: IF Date$ = "01-01-2000" THEN
3:     PRINT "Happy New Year"
4:     END IF
```

```

5: PRINT "The date is: "; Date$
6: PRINT "The time is: "; Time$
7: END

```

如果要使程序测试达到100%语句覆盖的目的,就只需执行将变量Date\$设为2000年1月1日的测试用例。程序将执行以下路径:

第1, 2, 3, 4, 5, 6, 7行

代码覆盖率分析器将声称每一条语句都得到了测试,实现了100%覆盖。可以退出测试了吗?不行!这样做虽然测试了所有语句,但是未测试所有分支。

直觉告诉我们还需要尝试日期不等于2000年1月1日的测试用例。这样在测试用例中,程序将执行其他路径:

第1, 2, 5, 6, 7行

大多数代码覆盖率分析器将根据代码分支,分别报告语句覆盖和分支覆盖的结果,使软件测试员更加清楚测试的效果。

7.5.3 条件覆盖

正当我们以为完事大吉时,却发现还有一个关于路径测试的复杂问题。清单7-3给出了一个与清单7-2略有不同的例子。第2行的IF增加了一个条件——同时检查日期和时间。条件覆盖(condition coverage)测试将分支语句的条件考虑在内。

清单7-3 IF语句中的多重条件生成了更多的代码路径

```

1: PRINT "Hello World"
2: IF Date$ = "01-01-2000" AND Time$ = "00:00:00" THEN
3:   PRINT "Happy New Year"
4:   END IF
5: PRINT "The date is: "; Date$
6: PRINT "The time is: "; Time$
7: END

```

在这个简单程序中,为了得到完整的条件覆盖测试,就需要增加如表7-2所示的4组测试用例。这些用例可以保证IF语句中的每一种可能都被覆盖。

表7-2 达到多重IF语句条件完全覆盖的测试用例

Date\$	Time\$	执行语句行
01-01-1999	11:11:11	1, 2, 5, 6, 7
01-01-1999	00:00:00	1, 2, 5, 6, 7
01-01-2000	11:11:11	1, 2, 5, 6, 7
01-01-2000	00:00:00	1, 2, 3, 4, 5, 6, 7

如果只考虑分支覆盖,前3个条件就是多余的,可以等价划分到一个测试用例中。但是,对于条件覆盖测试,所有4个案例都是重要的,因为它们用于测试第2行中IF语句的各种条件——错-错,错-对,对-错和对-对。

与分支覆盖一样,代码覆盖率分析器可以被设置为在报告结果时将条件考虑在内。如果

测试条件覆盖，就能达到分支覆盖，顺带也能达到语句覆盖。

注意 即使想方设法测试所有语句、分支和条件（除最简单的程序之外，这是不可能实现的），也还没有做到完全测试软件。不要忘了，本章第一部分讨论的数据错误仍然可能出现。程序流程和数据共同构成了软件的操作。

7.6 小结

本章说明了在程序运行的同时，访问软件的源代码可以开创一片全新的软件测试领域。动态白盒测试是非常强大的方法，通过提供关于测试对象的“内部”信息大幅减轻测试工作。了解代码的细节可以消除冗余的测试用例，增加针对原先没有考虑到的区间的测试用例。这两种方式都可以大幅提高测试的成效。

第4~7章讲述了软件测试的基础：

- 静态黑盒测试是指检查产品说明书，并在软件编写之前找出问题。
- 动态黑盒测试是指在不了解软件如何工作的前提下进行测试。
- 静态白盒测试是指通过正式审查和检验检查代码的细节。
- 动态白盒测试是指在看到软件的工作方式时，根据获得的信息对软件进行测试。

从某种意义上说，这就是软件测试的全部内容。当然，阅读这4章内容和在实际中运用是两回事。成为一个优秀的软件测试员需要有大量精力的投入和艰苦的努力。经过不断实践和经验的积累才能掌握如何合理地运用这些基本技术。

第三部分“运用测试技术”将讲述不同类型的软件测试以及在实际中如何应用“白盒和黑盒测试”技能。

7.7 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

1. 为什么了解了软件的工作方法会影响测试的方式和内容？
2. 动态白盒测试和调试有何区别？
3. 在大爆炸软件开发模式下几乎不可能进行测试的两个原因是什么？如何解决？
4. 判断是非：如果匆忙开发产品，就可以跳过模块测试而直接进行集成测试。
5. 测试桩和测试驱动有何差别？
6. 判断是非：总是首先设计黑盒测试用例。
7. 在本章描叙的三种代码覆盖中，哪一种最好？为什么？
8. 静态和动态白盒测试最大的问题是什么？